

ENTERPRISE KUBERNETES MASTERY

AI Platform Engineering Handbook

PART IX PLATFORM ENGINEERING

GitOps, Backstage, Crossplane, Progressive Delivery, Multi-Tenancy

Volume 9 of 16 Advanced Series

Prerequisites: Parts I through VIII

Edition 2025-2026

TABLE OF CONTENTS

1. Platform Engineering Philosophy and Goals	3
2. GitOps: Principles and Patterns	7
3. ArgoCD: GitOps Continuous Delivery	11
4. Flux: GitOps Toolkit	17
5. Helm: Package Management at Scale	21
6. Kustomize: Configuration Management	26
7. Backstage: Internal Developer Platform	30
8. Crossplane: Infrastructure as Code on Kubernetes .	36
9. Cluster Lifecycle Management	41
10. Multi-Tenancy Architectures	45
11. Progressive Delivery: Canary and Blue-Green	50
12. Landing Zones and Cluster Blueprints	55
13. Self-Service Platform Patterns	59
14. Platform Engineering for AI Workloads	63
15. Developer Experience Metrics	68
16. Platform Anti-Patterns	71
17. Hands-On Exercises	74

CHAPTER 1

Platform Engineering Philosophy and Goals

Platform Engineering emerged as a discipline in response to the complexity tax that Kubernetes imposed on development teams. The insight: Kubernetes is an excellent substrate for platform builders, but a terrible direct interface for application developers. Platform Engineering abstracts Kubernetes complexity into opinionated, self-service golden paths that improve developer velocity without sacrificing operational consistency or security.

The Platform Engineering Value Proposition

```
Without Platform Engineering: Developer -> learns Kubernetes API -> writes YAML -> debugs admission webhooks -> configures RBAC -> sets up monitoring -> configures HPA -> ... Time to production: weeks Consistency: zero (every team does it differently) Security: highly variable (depends on individual knowledge) With Platform Engineering: Developer -> uses self-service platform -> deploys via golden path -> monitoring, scaling, security enforced automatically Time to production: hours Consistency: high (platform enforces standards) Security: high (baked into the golden path)
```

Platform Engineering Core Principles

- * **Product thinking:** The platform is a product with developers as customers. Apply product management discipline: understand developer pain points, measure adoption, iterate based on feedback. Treat platform SLOs as product commitments.
- * **Golden paths, not golden cages:** Golden paths should be the path of least resistance to production. They should not be the only path. Escape hatches exist for teams with legitimate reasons to deviate.
- * **Self-service by default:** Developers should be able to deploy, scale, observe, and roll back their applications without filing tickets to platform or ops teams. Every manual approval is a tax on developer velocity.
- * **Paved roads:** The platform team builds and maintains the roads; application teams drive on them. Platform team owns reliability of the infrastructure layer; app teams own reliability of their applications.
- * **Shift left:** Security, compliance, and cost visibility belong in the developer workflow, not as post-deployment surprises. Admission controllers and IDE plugins surface issues before code reaches production.

Platform Team Topologies

Model	Structure	Best For	Risk
Centralised Platform Team	Single team owns all platform capabilities	Large orgs (500+ engineers)	Bottleneck if understaffed
Embedded Platform Engineers	Platform engineers embedded in product squads	Medium orgs	Inconsistent standards across squads
Platform as a Product	Dedicated product team with PM, engineers, UX	Large orgs with mature platform	Higher investment required

Model	Structure	Best For	Risk
Open Source Model	Platform team builds core; app teams contribute plugins	Very large orgs	Governance complexity

CHAPTER 2

GitOps: Principles and Patterns

GitOps is an operational model where Git is the single source of truth for both application code and infrastructure configuration. The desired state of the system is declared in Git; an automated agent (ArgoCD, Flux) continuously reconciles the actual state of the cluster to match. Changes are made via pull requests, not via direct `kubectl` commands.

GitOps Four Principles

- * **Declarative:** The entire system is described declaratively. No imperative scripts that are hard to audit, reproduce, or roll back.
- * **Versioned and immutable:** Desired state is stored in Git with full history. Every change is an atomic commit with a unique SHA. Rollback is a git revert.
- * **Pulled automatically:** Approved changes are applied automatically by a software agent without human intervention. No manual `kubectl` apply in production.
- * **Continuously reconciled:** Software agents continuously compare actual state to desired state and take corrective action. Drift is detected and corrected automatically.

GitOps Repository Patterns

Pattern	Structure	Pros	Cons
Mono-repo	All apps + all envs in one repo	Simple; atomic cross-app changes	Scales poorly; blast radius
App-per-repo	Each app has its own config repo	Good separation; team autonomy	Many repos to manage
Env-per-repo	Separate repo per environment	Clear env separation; branch protection	Cross-env PRs complex
Infra + App split	Infrastructure in separate repo from app configs	Clear ownership boundaries	Two-repo coordination required
Trunk-based + overlays	Single branch; env differences via Kustomize overlays	Simple branching; DRY configs	Kustomize complexity

Recommended GitOps Repository Structure

```

infrastructure-gitops/ clusters/ production-us-east/ cluster-config/ # Cluster-level config
(RBAC, policies) namespaces/ # Namespace definitions and ResourceQuotas apps/ # ArgoCD
ApplicationSets or Flux Kustomizations staging-us-east/ ... components/ monitoring/ #
Prometheus, Grafana stack security/ # Falco, cert-manager, Vault agent networking/ # Cilium
config, Gateway config storage/ # StorageClasses, Rook config app-gitops/ apps/ myapp/ base/ #
Base Kustomize manifests overlays/ dev/ # Dev-specific patches staging/ # Staging patches
  
```

```
production/ # Production patches helm/ # Helm values files per environment
```

CHAPTER 3

ArgoCD: GitOps Continuous Delivery

ArgoCD is the most widely deployed GitOps tool for Kubernetes. It continuously monitors Git repositories and automatically synchronises Kubernetes cluster state to match the desired state defined in Git. It supports Helm, Kustomize, Jsonnet, and raw YAML manifests.

ArgoCD Architecture

```
ArgoCD components: argocd-server: API server and Web UI (gRPC + REST) argocd-repo-server: Git
repository cloning and manifest generation argocd-application-controller: Reconciliation loop
(watches cluster vs Git) argocd-dex-server: OIDC identity provider (delegated auth)
argocd-redis: Cache for repo-server and app controller argocd-applicationset-controller:
Generates Applications from templates Reconciliation loop: 1. App controller fetches desired
state from repo-server (rendered manifests) 2. App controller queries Kubernetes API for live
state 3. Computes diff (OutOfSync if different) 4. If syncPolicy.automated: applies changes
automatically 5. Checks health of deployed resources 6. Reports status: Synced/OutOfSync +
Healthy/Degraded/Progressing
```

ArgoCD Application

```
apiVersion: argoproj.io/v1alpha1 kind: Application metadata: name: myapp-production namespace:
argocd finalizers: - resources-finalizer.argocd.argoproj.io # Cascade delete spec: project:
production source: repoURL: https://github.com/company/app-gitops targetRevision: main path:
apps/myapp/overlays/production destination: server: https://production-cluster.internal:6443
namespace: production syncPolicy: automated: prune: true # Delete resources removed from Git
selfHeal: true # Re-sync if cluster drifts from Git syncOptions: - CreateNamespace=true -
PrunePropagationPolicy=foreground - ApplyOutOfSyncOnly=true # Only apply changed resources
retry: limit: 5 backoff: duration: 5s maxDuration: 3m factor: 2 ignoreDifferences: - group:
apps kind: Deployment jsonPointers: - /spec/replicas # Ignore HPA-managed replica count
```

ApplicationSet -- Multi-Cluster Deployment

ApplicationSet generates multiple Applications from a template, enabling a single definition to deploy to many clusters or environments:

```
apiVersion: argoproj.io/v1alpha1 kind: ApplicationSet metadata: name: myapp-all-clusters
namespace: argocd spec: generators: # Deploy to all clusters registered in ArgoCD: - clusters:
selector: matchLabels: environment: production # Or: matrix of clusters x environments: -
matrix: generators: - clusters: values: region: us-east-1 - list: elements: - env: production
- env: staging template: metadata: name: myapp-{{name}} spec: project: default source:
repoURL: https://github.com/company/app-gitops path:
apps/myapp/overlays/{{metadata.labels.environment}} targetRevision: main destination: server:
'{{server}}' namespace: myapp syncPolicy: automated: { prune: true, selfHeal: true }
```


CHAPTER 4

Flux: GitOps Toolkit

Flux (CNCF graduated) is a GitOps toolkit composed of independent controllers for different aspects of the GitOps pipeline. Unlike ArgoCD (a monolithic CD tool), Flux is composable: each controller handles one concern, and they work together via Kubernetes events.

Flux Controllers

Controller	Responsibility	Key CRDs
source-controller	Fetches and caches Git repos, Helm charts, OCI artifacts, S3 buckets	GitRepository, HelmRepository, OCIRepository, Bucket
kustomize-controller	Applies Kustomize overlays from Sources	Kustomization
helm-controller	Manages Helm releases declaratively	HelmRelease
notification-controller	Sends events to Slack, GitHub, Teams; receives webhooks	Alert, Provider, Receiver
image-automation-controller	Auto-updates image tags in Git when new images pushed	ImageUpdateAutomation, ImagePolicy
image-reflector-controller	Scans registries for new image tags	ImageRepository

Flux GitRepository and Kustomization

```
# Source: watch Git repository: apiVersion: source.toolkit.fluxcd.io/v1 kind: GitRepository
metadata: name: app-gitops namespace: flux-system spec: interval: 1m url:
https://github.com/company/app-gitops ref: branch: main secretRef: name: github-credentials
--- # Apply: deploy production overlay from GitRepository: apiVersion:
kustomize.toolkit.fluxcd.io/v1 kind: Kustomization metadata: name: myapp-production namespace:
flux-system spec: interval: 5m path: ./apps/myapp/overlays/production prune: true sourceRef:
kind: GitRepository name: app-gitops healthChecks: - apiVersion: apps/v1 kind: Deployment
name: myapp namespace: production timeout: 5m retryInterval: 2m postBuild: substituteFrom: -
kind: ConfigMap name: cluster-vars # Inject cluster-specific variables
```

ArgoCD vs Flux Decision Matrix

Dimension	ArgoCD	Flux
Architecture	Monolithic CD platform	Composable GitOps toolkit
UI	Rich web UI included	CLI-first; UI via Weave GitOps (separate)
Multi-cluster	Built-in cluster management	Hub-spoke via Flux on each cluster

Dimension	ArgoCD	Flux
Helm support	First-class	HelmRelease CRD; feature-rich
ApplicationSet	Native (powerful templating)	Requires workarounds
Image automation	Via ArgoCD Image Updater (plugin)	Built-in image-automation-controller
Progressive delivery	Via Argo Rollouts	Via Flagger
Notification	Via notification plugin	Built-in notification-controller
Drift detection	Real-time (continuous watch)	Interval-based (configurable down to 1m)
Best for	Teams wanting full CD platform with UI	Teams wanting composable GitOps primitives

CHAPTER 5

Helm: Package Management at Scale

Helm is the package manager for Kubernetes. A Helm chart packages all Kubernetes manifests for an application with configurable values, lifecycle hooks, and dependency management. At enterprise scale, Helm requires disciplined chart design, values management, and integration with GitOps tooling.

Helm Chart Structure

```
myapp/ Chart.yaml # Chart metadata (name, version, appVersion, dependencies) values.yaml #
Default values (all configurable options documented) values-schema.json # JSON Schema for
values validation charts/ # Dependency charts (pulled by helm dependency update) templates/
_helpers.tpl # Named templates (reusable snippets) deployment.yaml # Deployment template
service.yaml # Service template ingress.yaml # Ingress template (conditional) hpa.yaml # HPA
template (conditional) configmap.yaml # ConfigMap template serviceaccount.yaml rbac.yaml
pdb.yaml # PodDisruptionBudget networkpolicy.yaml NOTES.txt # Post-install instructions ci/
test-values.yaml # Values for CI testing production-values.yaml tests/ connection-test.yaml #
Helm test Pod
```

Production Helm Values Pattern

```
# values.yaml (chart defaults -- safe, minimal): replicaCount: 1 image: repository:
harbor.internal.corp/myapp tag: latest # Overridden in CI/CD pullPolicy: IfNotPresent
resources: requests: cpu: 100m memory: 128Mi limits: memory: 256Mi autoscaling: enabled: false
minReplicas: 1 maxReplicas: 10 targetCPUUtilizationPercentage: 70 podSecurityContext:
runAsNonRoot: true runAsUser: 10001 fsGroup: 10001 seccompProfile: type: RuntimeDefault
securityContext: allowPrivilegeEscalation: false readOnlyRootFilesystem: true capabilities:
drop: [ALL] pdb: enabled: false minAvailable: 1 # values-production.yaml (production
overrides): replicaCount: 3 image: tag: v1.2.3 # Set by CI/CD pipeline autoscaling: enabled:
true minReplicas: 3 maxReplicas: 50 pdb: enabled: true minAvailable: 2
```

CHAPTER 6

Kustomize: Configuration Management

Kustomize is a template-free configuration customisation tool built into kubectl. It allows defining a base set of manifests and applying environment-specific overlays (patches, transformers, generators) without templates or variable substitution. This makes base manifests always valid Kubernetes YAML.

Kustomize Architecture

```
base/ kustomization.yaml # Lists all resources deployment.yaml # Production-ready base
deployment service.yaml configmap.yaml overlays/ dev/ kustomization.yaml # References base +
patches replica-patch.yaml # Patch: replicas=1 for dev resource-patch.yaml # Patch: smaller
resource requests staging/ kustomization.yaml replica-patch.yaml # Patch: replicas=2
production/ kustomization.yaml replica-patch.yaml # Patch: replicas=5 hpa.yaml # Additional
resource: HPA pdb.yaml # Additional resource: PDB
```

Kustomization Files

```
# base/kustomization.yaml: apiVersion: kustomize.config.k8s.io/v1beta1 kind: Kustomization
resources: - deployment.yaml - service.yaml - configmap.yaml commonLabels:
app.kubernetes.io/managed-by: kustomize images: - name: myapp newName:
harbor.internal.corp/myapp newTag: v1.0.0 # overlays/production/kustomization.yaml:
apiVersion: kustomize.config.k8s.io/v1beta1 kind: Kustomization namespace: production
resources: - ../../base - hpa.yaml - pdb.yaml images: - name: myapp newTag: v1.2.3 # Override
image tag patches: - patch: |- - op: replace path: /spec/replicas value: 5 target: kind:
Deployment name: myapp configMapGenerator: - name: app-config behavior: merge literals: -
LOG_LEVEL=warn - ENVIRONMENT=production
```

CHAPTER 7

Backstage: Internal Developer Platform

Backstage (Spotify, donated to CNCF) is the most widely adopted framework for building Internal Developer Platforms. It provides a plugin-based architecture for building a unified developer portal: service catalogue, software templates, documentation, and integrations with every tool in the developer ecosystem.

Backstage Core Components

- * **Software Catalogue:** The living inventory of all software components, APIs, teams, resources, and systems in the organisation. Every service has a `catalog-info.yaml` that describes ownership, documentation, dependencies, and deployment status. Powers discovery: developers can find who owns a service, what APIs it exposes, and how to contact the team.
- * **Software Templates (Scaffolder):** Golden path templates that developers use to create new services. A template encodes all organisational standards: repository structure, CI/CD pipeline, Kubernetes manifests, monitoring config, security scanning, RBAC -- all pre-configured and policy-compliant. New service ready in minutes, not weeks.
- * **TechDocs:** Documentation-as-code: Markdown docs stored alongside the service code, rendered and indexed by Backstage. Docs are discoverable in the catalogue alongside the service. Eliminates stale wiki documentation.
- * **Search:** Unified search across catalogue, docs, APIs, and all integrated tools. Developers find information without knowing which system it lives in.
- * **Plugins:** Over 200 community plugins integrate Backstage with: GitHub, GitLab, Jenkins, ArgoCD, Kubernetes, Prometheus, Grafana, PagerDuty, Datadog, Vault, SonarQube, Snyk, and hundreds more.

catalog-info.yaml -- Service Catalogue Entry

```
apiVersion: backstage.io/v1alpha1 kind: Component metadata: name: inference-api description:
LLM inference API for enterprise AI platform labels: tier: ai-serving team: ai-platform
annotations: github.com/project-slug: company/inference-api backstage.io/techdocs-ref: dir:.
argocd/app-name: inference-api-production grafana/dashboard-selector: 'title=inference-api'
pagerduty.com/service-id: PXXXXXX sonarqube.org/project-key: inference-api tags: - ai -
production - python spec: type: service lifecycle: production owner: group:ai-platform-team
system: ai-platform providesApis: - inference-api-v2 dependsOn: - component:vector-database -
component:model-registry - resource:gpu-cluster
```

Backstage Software Template for AI Service

```
# template.yaml (new AI inference service golden path): apiVersion:
scaffolder.backstage.io/v1beta3 kind: Template metadata: name: ai-inference-service title: AI
Inference Service description: Deploy a new LLM inference endpoint on Kubernetes tags: [ai,
kubernetes, python] spec: owner: ai-platform-team type: service parameters: - title: Service
Details required: [name, model, team] properties: name: { type: string, title: Service Name }
model: type: string title: Base Model enum: [llama-3-8b, llama-3-70b, mistral-7b, custom]
gpuCount: type: integer title: Number of GPUs default: 1 minimum: 1 maximum: 8 team: { type:
string, title: Owning Team } steps: - id: fetch name: Fetch Template action: fetch:template
input: url: ./skeleton values: name: '${{ parameters.name }}' model: '${{ parameters.model }}'
```

```
gpuCount: '${{ parameters.gpuCount }}' - id: create-repo name: Create GitHub Repository  
action: publish:github input: repoUrl: github.com?owner=company&repo;=${{ parameters.name }} -  
id: argocd-app name: Register ArgoCD Application action: argocd:create-resources input:  
appName: '${{ parameters.name }}-production' projectName: ai-serving
```

CHAPTER 8

Crossplane: Infrastructure as Code on Kubernetes

Crossplane (CNCF graduated) extends Kubernetes to manage cloud infrastructure resources using the same declarative API model used for application workloads. It enables platform teams to define Composite Resources -- opinionated abstractions over cloud services -- that developers consume via familiar Kubernetes APIs.

Crossplane Architecture

```
Crossplane components: Crossplane core: Controller that manages XR composition
Provider: Plugin that maps K8s resources to cloud API calls
provider-aws: AWS resources (RDS, S3, ElastiCache, EKS...)
provider-gcp: GCP resources (CloudSQL, GCS, GKE...)
provider-azure: Azure resources (AKS, Cosmos DB, Storage...)
CompositeResourceDefinition (XRD): Defines a new API type (like CRD)
Composition: Maps the XR to concrete managed resources
Composite Resource (XR): Cluster-scoped instance of an XRD
Claim: Namespace-scoped reference to an XR (for developers)
Request flow: Developer creates Claim -> XR created -> Composition creates managed resources -> Provider calls cloud API -> Cloud resource created -> Status propagated back
```

Crossplane: Self-Service Database

```
# Platform team defines XRD (the API that developers use):
apiVersion: apiextensions.crossplane.io/v1 kind: CompositeResourceDefinition metadata: name:
xpostgresinstances.db.company.com spec: group: db.company.com names: kind:
XPostgreSQLInstance plural: xpostgresinstances claimNames: kind: PostgreSQLInstance # What
developers use plural: postgresqlinstances versions: - name: v1alpha1 served: true
referenceable: true schema: openAPIV3Schema: type: object properties: spec: type: object
properties: parameters: type: object properties: storageGB: type: integer default: 20 tier:
type: string enum: [dev, staging, production] --- # Developer creates a Claim (no cloud
knowledge required):
apiVersion: db.company.com/v1alpha1 kind: PostgreSQLInstance metadata:
name: myapp-db namespace: production spec: parameters: storageGB: 100 tier: production
writeConnectionSecretToRef: name: myapp-db-credentials
```

CHAPTER 9

Cluster Lifecycle Management

Enterprise Kubernetes fleets require automated cluster provisioning, upgrading, scaling, and decommissioning. Manual cluster management does not scale beyond a handful of clusters. ClusterAPI and managed Kubernetes services provide the declarative cluster lifecycle management that mirrors application GitOps.

Cluster Management Tools

Tool	Approach	Multi-Cloud	Best For
ClusterAPI (CAPI)	Declarative cluster lifecycle via K8s API	Yes (providers for AWS, GCP, Azure, vSphere, bare metal)	Standardised cluster provisioning across environments
EKS (eksctl/Terraform)	AWS-managed control plane	AWS only	AWS-native deployments
GKE (Terraform/Config Connector)	GCP-managed control plane	GCP only	GCP-native; GKE Autopilot for serverless nodes
AKS (Terraform/Bicep)	Azure-managed control plane	Azure only	Azure-native; Arc for hybrid
Rancher	Multi-cluster management UI + RKE/RKE2	Yes	On-premises + multi-cloud; edge clusters
Tanzu (VMware/Broadcom)	vSphere-integrated K8s	VMware + cloud	VMware-centric enterprises
OpenShift	Full enterprise K8s platform	Yes	Enterprises needing integrated platform

Cluster Upgrade Strategy

```
Kubernetes releases 3 minor versions per year. Support window: latest 3 minor versions.
Enterprise recommendation: stay within 1-2 minor versions of latest. UPGRADE ORDER (must be
followed): 1. etcd (if managing self) 2. Control plane nodes (one at a time for HA) 3. Worker
nodes (rolling drain + upgrade) PRE-UPGRADE CHECKLIST: Backup etcd (mandatory) Review API
deprecations (kubectl convert, kube-no-trouble) Test upgrade in staging first Check addon
compatibility (CNI, CSI, ingress versions) Verify PodDisruptionBudgets protect critical
workloads # Check for deprecated APIs before upgrade: kubectl krew install deprecations
kubectl deprecations --k8s-version v1.32 # Drain node for upgrade: kubectl drain node-01 \
--ignore-daemonsets \ --delete-emptydir-data \ --timeout=300s # Upgrade node OS/k8s binaries
```



```
kubectrl uncordon node-01
```

CHAPTER 10

Multi-Tenancy Architectures

Multi-tenancy in Kubernetes means multiple teams or customers sharing cluster infrastructure. The right isolation model depends on the trust level between tenants and the regulatory requirements.

Multi-Tenancy Models

Model	Isolation	Trust Level	Resource Efficiency	Use Case
Cluster per tenant	Maximum (separate cluster)	Untrusted	Low (cluster overhead per tenant)	High-compliance SaaS, regulated industries
Namespace per tenant	Moderate (RBAC + NetworkPolicy + Quota)	Semi-trusted	High	Internal teams, same organisation
vCluster per tenant	Strong (virtual cluster in namespace)	Semi-trusted	Medium	Dev environments, CI namespaces
HierarchicalNamespace	Moderate (namespace hierarchy)	Semi-trusted	High	Large orgs with sub-teams
Pod-level isolation (Kata)	Hypervisor boundary	Low-trust workloads	Medium	Multi-tenant inference serving

vCluster -- Virtual Clusters

vCluster creates lightweight virtual Kubernetes clusters inside namespaces. Each tenant gets a dedicated API server (k3s or k0s), control plane, and RBAC -- all running inside a single namespace on the host cluster:

```
# Create a virtual cluster for a development team: vcluster create dev-team-a \ --namespace
vcluster-dev-team-a \ --chart-version 0.20.0 \ --values - <<'YAML' controlPlane: distro: k3s:
enabled: true statefulSet: resources: requests: { cpu: 200m, memory: 256Mi } sync: toHost:
ingresses: enabled: true YAML # Connect to the virtual cluster: vcluster connect dev-team-a
--namespace vcluster-dev-team-a # Team sees a fully isolated K8s cluster: kubectrl get nodes #
Shows virtual node kubectrl get ns # Clean namespace list
```

CHAPTER 11

Progressive Delivery: Canary and Blue-Green

Progressive delivery extends GitOps by controlling how a new version is gradually released to production: starting with a small percentage of traffic, measuring quality signals, and automatically promoting or rolling back based on metrics. This reduces deployment risk to near zero.

Argo Rollouts -- Progressive Delivery Controller

Argo Rollouts extends Kubernetes Deployments with canary and blue-green strategies, integrated with Service Mesh and ingress controllers for traffic splitting:

```
apiVersion: argoproj.io/v1alpha1 kind: Rollout metadata: name: api-server spec: replicas: 10
selector: matchLabels: { app: api-server } template: metadata: labels: { app: api-server }
spec: containers: - name: api image: harbor.corp/api:v1.2.3 strategy: canary: canaryService:
api-server-canary stableService: api-server-stable trafficRouting: istio: virtualService:
name: api-server-vs steps: - setWeight: 5 # 5% to canary - pause: { duration: 5m } - analysis:
# Automated quality gate templates: - templateName: success-rate - setWeight: 20 - pause: {
duration: 10m } - analysis: templates: - templateName: latency-check - setWeight: 50 - pause:
{ duration: 10m } - setWeight: 100 --- apiVersion: argoproj.io/v1alpha1 kind: AnalysisTemplate
metadata: name: success-rate spec: metrics: - name: success-rate successCondition: result[0]
>= 0.99 failureCondition: result[0] < 0.95 provider: prometheus: address:
http://prometheus:9090 query: |
sum(rate(http_requests_total{status!~'5..',app='api-server',track='canary'}[5m])) /
sum(rate(http_requests_total{app='api-server',track='canary'}[5m]))
```

CHAPTER 12

Landing Zones and Cluster Blueprints

A landing zone is a pre-configured, security-hardened, compliance-ready environment that serves as the starting point for new clusters or new teams. Cluster blueprints encode all organisational standards as code, enabling new clusters to be spun up that are immediately production-ready.

Cluster Bootstrap Architecture

```
Cluster bootstrap sequence (ArgoCD App of Apps pattern): 1. Cluster provisioned (CAPI / EKS / GKE / AKS) 2. ArgoCD installed (helm install argocd) 3. Bootstrap Application applied: kubectl apply -f bootstrap-app.yaml 4. Bootstrap App deploys App of Apps: app-of-apps/ networking.yaml -> installs Cilium, Gateway security.yaml -> installs cert-manager, Vault agent, Falco observability.yaml -> installs Prometheus stack, Loki, Tempo storage.yaml -> installs CSI drivers, StorageClasses policy.yaml -> installs Kyverno + baseline policies platform.yaml -> installs Backstage, Crossplane cluster-config.yaml -> RBAC, Namespaces, ResourceQuotas 5. All platform components become healthy 6. Cluster registered in Backstage catalogue 7. Developers can immediately self-serve
```

Standard Namespace Labels and Annotations

```
# Every production namespace created with this template: apiVersion: v1 kind: Namespace metadata: name: team-platform labels: team: platform cost-center: CC-1234 environment: production tier: platform # Pod Security Standards: pod-security.kubernetes.io/enforce: restricted pod-security.kubernetes.io/enforce-version: v1.30 pod-security.kubernetes.io/warn: restricted # Istio mesh participation: istio-injection: enabled # Network policy automation: network-policy.company.com/default-deny: enabled annotations: company.com/team-contact: platform@company.com company.com/pagerduty-service: PXXXXXX company.com/runbook: https://wiki.company.com/platform
```

CHAPTER 13

Self-Service Platform Patterns

Self-service is the core value proposition of Platform Engineering. Every manual approval process, every ticket to ops, every JIRA to get a namespace created is a tax on developer velocity. The platform should enable developers to do everything they need for their application without waiting.

Self-Service Capabilities Matrix

Capability	Manual Process	Self-Service Implementation	Time Saving
New namespace	File ticket to ops	Backstage template + Crossplane	Days -> Minutes
Database provisioning	DBA ticket	Crossplane XR claim	Days -> Hours
TLS certificate	PKI team request	cert-manager auto-issue	Days -> Seconds
Secret access	Security team approval	Vault policy + ESO	Days -> Minutes
Deploy to production	Manual approval + ops deploy	ArgoCD sync (automated)	Hours -> Minutes
Scale up	Ops team intervention	HPA / KEDA auto-scaling	Hours -> Seconds
Add team member	Admin ticket	OIDC group + RBAC sync	Days -> Self-service
New cluster (dev)	Infra team request	vCluster from Backstage template	Days -> Minutes

Namespace-as-a-Service Pattern

```
# Crossplane Composition for self-service namespace: # Developer creates: NamespaceClaim #
Platform creates: Namespace + RBAC + ResourceQuota + NetworkPolicy + Vault path apiVersion:
platform.company.com/v1alpha1 kind: NamespaceClaim metadata: name: myteam-dev spec: team:
myteam environment: development costCenter: CC-5678 cpuQuota: '20' memoryQuota: 40Gi gpuQuota:
'4' # Composition automatically creates: # 1. Namespace with standard labels and PSS
enforcement # 2. RBAC RoleBindings for team OIDC group # 3. ResourceQuota with requested
limits # 4. LimitRange with defaults # 5. NetworkPolicy default-deny + DNS egress allow # 6.
Vault namespace + KV path for team secrets # 7. Backstage catalogue registration
```

CHAPTER 14

Platform Engineering for AI Workloads

AI workloads place unique demands on the platform that standard application platform patterns do not address. GPU resource management, model lifecycle, experiment tracking, and the scale of compute required for LLM training all require platform-level abstractions beyond standard Kubernetes.

AI Platform Golden Paths

- * **LLM inference deployment:** One-click deployment of a model to KServe/vLLM. Developer selects model from registry, sets GPU count, replica count, and autoscaling policy. Platform provisions: Deployment + Service + Gateway HTTPRoute + HPA/KEDA + Prometheus metrics + Grafana dashboard + PagerDuty alert.
- * **Training job submission:** Self-service ML training job submission. Developer fills form: dataset path, model config, GPU count, max runtime. Platform submits: Volcano Job + PVC for checkpoint + MLflow tracking setup + budget alert if compute exceeds threshold.
- * **RAG pipeline deployment:** Golden path for RAG architecture: vector database + embedding service + LLM + retrieval API. Template provisions all components with correct networking, security, and observability.
- * **Experiment namespace:** Isolated environment for ML experiments. Short-lived (72h TTL) namespace with GPU quota, MLflow tracking, object storage access, and Jupyter notebook server.

GPU Resource Management

```
GPU resources require special platform-level management: NVIDIA GPU Operator (platform
component, DaemonSet): Installs: NVIDIA drivers, container toolkit, device plugin, DCGM
exporter Enables: nvidia.com/gpu resource type for scheduling GPU ResourceQuota per team
namespace: requests.nvidia.com/gpu: 8 limits.nvidia.com/gpu: 8 (GPU requests == limits;
fractional GPU not supported in standard K8s) MIG (Multi-Instance GPU) for shared GPU access:
NVIDIA A100/H100 support splitting GPU into up to 7 instances Each instance is an isolated GPU
with dedicated memory Useful for small models or development workloads # Enable MIG on a node:
nvidia-smi mig -cgi 1g.10gb,1g.10gb,1g.10gb -C # MIG resources appear as:
nvidia.com/mig-1g.10gb: 3 # 3 x 10GB MIG instances available GPU observability (DCGM Exporter
metrics): DCGM_FI_DEV_GPU_UTIL: GPU utilisation % DCGM_FI_DEV_MEM_COPY_UTIL: Memory bandwidth
utilisation % DCGM_FI_DEV_FB_USED: GPU memory used (bytes) DCGM_FI_DEV_POWER_USAGE: Power
consumption (watts) DCGM_FI_DEV_SM_CLOCK: SM clock frequency (MHz)
```

CHAPTER 15

Developer Experience Metrics

Platform Engineering success must be measured. DORA metrics (from the DevOps Research and Assessment program) are the industry-standard framework for measuring software delivery performance, and platform engineering directly impacts all four.

DORA Metrics and Kubernetes Indicators

DORA Metric	Definition	Elite Target	K8s/Platform Indicator
Deployment Frequency	How often deployments to production	On demand (multiple/day)	ArgoCD sync frequency; image update automation
Lead Time for Changes	Code commit to production	Less than 1 hour	CI pipeline + ArgoCD sync time
Change Failure Rate	% deployments causing incidents	Less than 5%	Argo Rollout failures; incident rate post-deploy
Failed Deployment Recovery Time	Time to restore after failure	Less than 1 hour	Rollback time via Argo Rollouts

Platform-Specific Metrics

- * **Time to First Deployment:** How long from developer onboarding to their first successful production deployment. Target: under 1 day with Backstage golden paths.
- * **Self-service ratio:** Percentage of infrastructure changes made via self-service vs tickets. Target: greater than 90%. Measures platform adoption and ticket reduction.
- * **Platform SLA:** Uptime and latency of platform components (API server, ArgoCD, Backstage, Vault). Platform is now critical infrastructure; it needs SLOs and alerting.
- * **Cognitive load score:** Developer survey metric: how much mental overhead does working with the platform impose? Target: decrease over time with each platform improvement.

CHAPTER 16

Platform Anti-Patterns

Anti-Pattern: Building a platform nobody uses

Problem: Platform team builds features based on assumptions, not developer needs. Low adoption because it does not solve real pain points.

Solution: Start with developer interviews. Build the simplest useful thing first. Measure adoption monthly. Run quarterly developer experience surveys.

Anti-Pattern: Golden cage (not golden path)

Problem: Platform so restrictive that developers cannot do legitimate work without workarounds. Developers bypass the platform entirely (shadow IT).

Solution: Provide escape hatches for legitimate deviations. Document the process. Make the golden path the easiest path, not the only path.

Anti-Pattern: GitOps without drift detection

Problem: Using ArgoCD with `selfHeal: false`. Developers make ad-hoc `kubectl` changes that drift from Git, causing inconsistency and failed deployments.

Solution: Enable `selfHeal: true` and `prune: true` in ArgoCD. Enforce cluster access via RBAC: only ArgoCD SA has write access to production namespaces.

Anti-Pattern: Helm chart sprawl

Problem: Hundreds of bespoke Helm charts, each slightly different. No shared base charts. Security defaults not enforced.

Solution: Define 2-3 standard chart patterns (web service, worker, cronjob). Enforce via OPA/Kyverno. Use Helm library charts for shared templates.

Anti-Pattern: Ignoring platform reliability

Problem: ArgoCD, cert-manager, Vault deployed without HA, no PDB, no monitoring. Platform outage blocks all deployments cluster-wide.

Solution: Platform components are critical infrastructure. HA, PDB, Prometheus alerts, SLOs, and on-call coverage required for all platform components.

Anti-Pattern: Manual cluster creation

Problem: Clusters created via console clicks with no IaC. No two clusters are identical. Reproducing configuration is impossible.

Solution: All clusters created via Terraform/CAPI + ArgoCD bootstrap. Cluster creation should be a PR to a GitOps repository.

CHAPTER 17

Hands-On Exercises

Exercise 9.1 -- GitOps with ArgoCD

Deploy an application using GitOps principles:

```
# Install ArgoCD: kubectl create namespace argocd kubectl apply -n argocd -f
https://raw.githubusercontent.com/argoproj/argo-cd/stable/manifests/install.yaml kubectl wait
--for=condition=available deployment/argocd-server -n argocd --timeout=5m # Get admin
password: kubectl -n argocd get secret argocd-initial-admin-secret \ -o
jsonpath='.data.password' | base64 -d # Port-forward ArgoCD UI: kubectl port-forward
svc/argocd-server -n argocd 8080:443 # Create an Application pointing to a Git repo: argocd
login localhost:8080 --username admin --insecure argocd app create guestbook \ --repo
https://github.com/argoproj/argocd-example-apps.git \ --path guestbook \ --dest-server
https://kubernetes.default.svc \ --dest-namespace guestbook \ --sync-policy automated \
--auto-prune --self-heal # Observe reconciliation: argocd app get guestbook argocd app history
guestbook
```

Exercise 9.2 -- Kustomize Overlay Pipeline

Create a multi-environment deployment with Kustomize:

```
# Create base manifests: mkdir -p myapp/base myapp/overlays/{dev,production} #
base/deployment.yaml: replicas=1, image=nginx:alpine # base/kustomization.yaml: resources:
[deployment.yaml] # overlays/dev/kustomization.yaml: # resources: [../../base] # patches:
(replicas=1, small resources) # overlays/production/kustomization.yaml: # resources:
[../../base] # patches: (replicas=5, larger resources) # commonLabels: {environment:
production} # Preview what would be deployed: kubectl kustomize myapp/overlays/dev kubectl
kustomize myapp/overlays/production # Deploy development overlay: kubectl apply -k
myapp/overlays/dev # Deploy production overlay: kubectl apply -k myapp/overlays/production
```

End of Part IX -- Continue to Part X: Observability

Part X covers the complete observability stack: OpenTelemetry instrumentation, Prometheus and Thanos for metrics at scale, Grafana dashboards and alerting, Loki for log aggregation, Tempo for distributed tracing, Jaeger, Fluent Bit, Elastic, cost observability with OpenCost, GPU observability with DCGM, and AI-specific observability for LLM inference monitoring.